

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	B.V.N.S.BHIMESWAR	Reg. No.:	192472048
Section / Batch:	CSE AI	Date:	11/08/26

Code of Conduct

*I **B.V.N.S.BHIMESWAR / 192472048** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____ **B.V.N.S.BHIMESWAR** _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. A company is developing an AI-powered customer support chatbot for banking services. The chatbot uses Context-Free Grammar (CFG) for parsing user queries. However, it faces issues such as:

Incorrect interpretation of ambiguous sentences

Failure to handle subject–verb agreement

Inefficient parsing for long queries Example user query:

“Show me the transactions with the card from last month”

- Analyze the ambiguity present in the given sentence using CFG
- Evaluate the limitations of CFG in handling real-world conversational queries
- Evaluate and propose an improved solution using PCFG, feature structures, or Earley parsing
- Justify how your proposed method improves accuracy and efficiency in an industry setting

2. A voice assistant system (like Alexa/Siri-type applications) processes spoken commands using parsing techniques. The system currently uses top-down parsing, but faces:

Backtracking issues

Poor handling of incomplete or ambiguous input

Difficulty in real-time response Example command:

“Book a flight to Delhi with a window seat”

- a) Analyze how the given sentence can lead to multiple parse structures
- b) Analyze the limitations of top-down parsing in this real-time application
- c) Analyze how Earley parsing can improve handling of ambiguity and partial inputs
- d) Compare and analyze the performance of top-down vs Earley parsing for such industry use cases

3. A healthcare company is developing an AI system to analyze patient medical reports and automatically extract critical information such as diagnosis, prescribed treatments, and follow-up actions.

The system must handle:

Complex and lengthy medical sentences

Ambiguity in sentence structure

Accurate subject-verb agreement

Relationships between medical actions and entities (sub-categorization) Real-time processing for hospital use Example input:

"The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai."

The current system, based only on basic Context-Free Grammar (CFG), fails to correctly interpret relationships and produces inconsistent outputs.

- a) Design a complete NLP architecture integrating: CFG for syntactic structure

Probabilistic CFG (PCFG) for ambiguity resolution

Feature Structures for agreement handling

Efficient parsing techniques

- b) Develop a step-by-step workflow showing how the system processes the given medical sentence from input to structured output (diagnosis, action, location).
- c) Create a method to:

Incorporate sub-categorization frames for medical verbs

Handle real-time data processing

Improve accuracy and scalability in a hospital environment

TABLE OF CONTENTS

Q1. Banking Chatbot – CFG Ambiguity and Improved Parsing

Q2. Voice Assistant – Top-Down vs Earley Parsing

Q3. Healthcare NLP – Integrated Semantic Processing Architecture

Overall Conclusion

Q1. Banking Chatbot – CFG Ambiguity and Improved Parsing

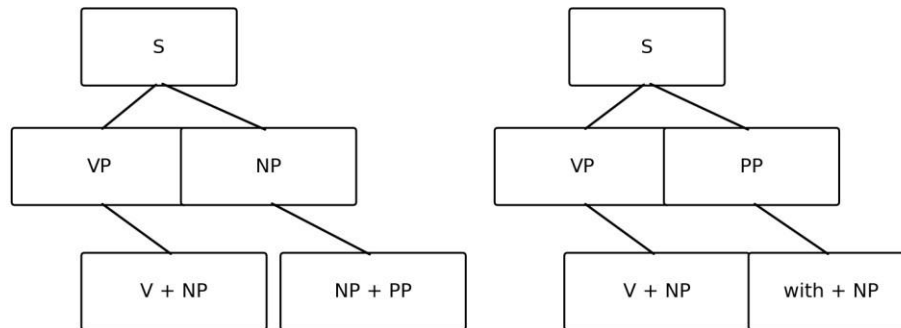
The industry scenario describes an AI-powered banking customer-support chatbot using Context-Free Grammar (CFG). The stated problems are incorrect interpretation of ambiguous sentences, failure to handle subject–verb agreement, and inefficient parsing for long queries. The example query is: “Show me the transactions with the card from last month.”

(a) Analyze the ambiguity using CFG

The sentence contains attachment ambiguity. The prepositional phrase “with the card” can be interpreted as modifying the noun phrase “transactions” or as being associated with the action “show”. The phrase “from last month” also requires attachment to the intended part of the sentence.

“Show me the transactions with the card from last month”

Parse A: “with the card” modifies TRANSACTION **Parse B: “with the card” attaches to the ACTION**



“from last month” is also an attachment point; a probabilistic model can rank the intended reading.

A simplified CFG-style analysis can represent the two readings as follows:

- Reading 1: [transactions [with the card]] [from last month] — the card identifies the transactions.
- Reading 2: [show the transactions] [with the card] [from last month] — the PP is treated as an action-related modifier.
- The grammar can generate more than one syntactic structure without encoding which interpretation is most likely.

Industry interpretation

For a banking chatbot, the intended meaning should normally be the transaction set associated with the card and restricted to the previous month. The system therefore needs semantic and contextual information in addition to CFG.

(b) Evaluate limitations of CFG for real-world conversational queries

- CFG represents grammatical structure but does not inherently assign probabilities to competing parses.

- Ambiguous attachment can generate multiple valid parse trees without selecting the intended meaning.
- Basic CFG does not naturally encode subject–verb agreement unless grammar rules are expanded with agreement-specific variants.
- Long conversational queries can produce a large number of possible parse combinations.
- Conversational language may be incomplete, elliptical, informal or context dependent.
- A purely syntactic grammar does not by itself resolve user intent, domain entities or semantic relationships.

Key limitation

CFG is useful for structural parsing, but the banking application requires probability, agreement features, semantic constraints and efficient parsing to make a reliable decision.

(c) Evaluate and propose an improved solution

A practical improvement is to combine CFG with feature structures, PCFG and an efficient parser such as Earley parsing. Each component addresses a different weakness.

Component	Purpose	Benefit
CFG	Generate basic syntactic structures	Provides grammatical foundation
Feature Structures	Represent number/person/agreement and other attributes	Reduces agreement errors
PCFG	Assign probabilities to alternative parses	Ranks competing interpretations
Earley Parsing	Efficient chart parsing for general CFGs	Avoids unnecessary repeated work and supports complex/partial structures

(d) Justification for accuracy and efficiency

The improved approach first generates structurally valid parses, uses feature constraints to reject inconsistent agreement, and then uses probabilistic ranking to prefer the interpretation that best fits learned banking-language patterns. Earley chart parsing stores intermediate states so that repeated subproblems do not need to be recomputed.

- Higher semantic accuracy through probabilistic ranking and contextual evidence.
- Fewer agreement-related errors through feature structures.
- Better handling of long and complex queries through chart-based parsing.
- More consistent banking-domain interpretation when trained with domain-specific data.
- A modular design allows grammar, probabilities and semantic rules to be improved independently.

Recommended industry solution

CFG + Feature Structures + PCFG ranking + Earley parsing, followed by semantic interpretation and banking-domain validation, provides a stronger solution than basic CFG alone.

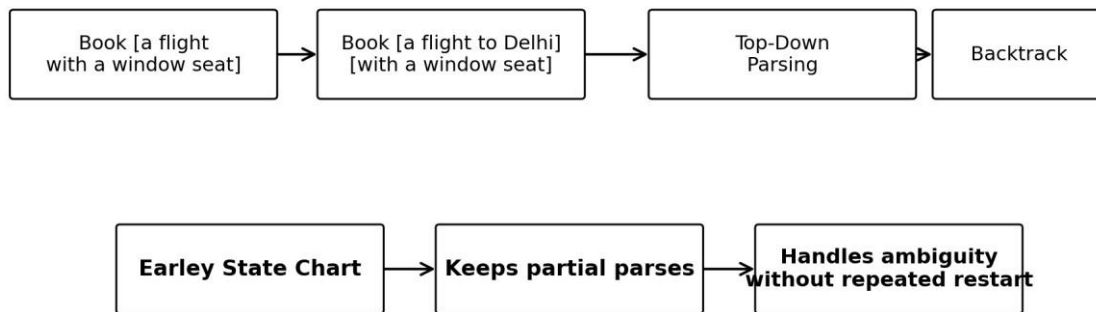
Q2. Voice Assistant – Top-Down Parsing vs Earley Parsing

The voice-assistant scenario uses top-down parsing but faces backtracking, poor handling of incomplete or ambiguous input, and difficulty producing real-time responses. The example command is: “Book a flight to Delhi with a window seat.”

(a) Analyze multiple parse structures

The phrase “with a window seat” creates a prepositional-phrase attachment decision. One interpretation treats the phrase as part of the requested flight description; another treats it as an additional booking constraint associated with the booking action.

“Book a flight to Delhi with a window seat”



Earley parsing is well suited to incomplete input and real-time ambiguity management.

- Parse 1: Book [a flight to Delhi with a window seat]. Here, “with a window seat” is part of the flight description.
- Parse 2: Book [a flight to Delhi] [with a window seat]. Here, the phrase is an additional constraint on the booking action.
- The semantic result is closely related in this example, but the structural ambiguity demonstrates why a real-time parser must maintain alternative interpretations until enough evidence is available.

(b) Limitations of top-down parsing

- It begins from the start symbol and predicts productions before seeing all required input evidence.
- Ambiguous grammar choices can cause repeated backtracking.
- Incomplete spoken commands may leave the parser with unfinished hypotheses.
- Multiple alternatives can increase computation and response latency.
- The method can be inefficient when many possible derivations share common partial structures.

Real-time concern

A voice assistant must produce a response quickly. Repeated backtracking can increase latency, particularly when speech recognition produces uncertain, partial or ambiguous input.

(c) How Earley parsing improves ambiguity and partial-input handling

Earley parsing is a chart-based parsing method that records partial states as the input is processed. Its three core operations are commonly described as prediction, scanning and completion.

1. Prediction: introduce grammar rules that may expand from the current state.
2. Scanning: consume a matching input token and advance the state.
3. Completion: when a constituent is completed, update states that were waiting for it.

Because intermediate states are retained in a chart, the parser can represent several hypotheses without restarting the entire parse. This makes it suitable for ambiguous and incomplete input in a voice-assistant setting.

(d) Performance comparison

Ambiguity	May require backtracking Can leave incomplete	Maintains alternative states Naturally maintains derivations difficult to partial chart states commands	Earley is better for competing parses Useful for spoken
Partial input			partial chart
	manage		
Repeated work	Can repeat exploration during backtracking	Shares intermediate states	Better computational reuse
	Latency may increase	More systematic chart	More suitable for
Real-time response	with alternatives	processing	complex real-time parsing
Grammar flexibility	Works well with controlled grammars	Handles general CFG- style structures	Useful for broad language coverage

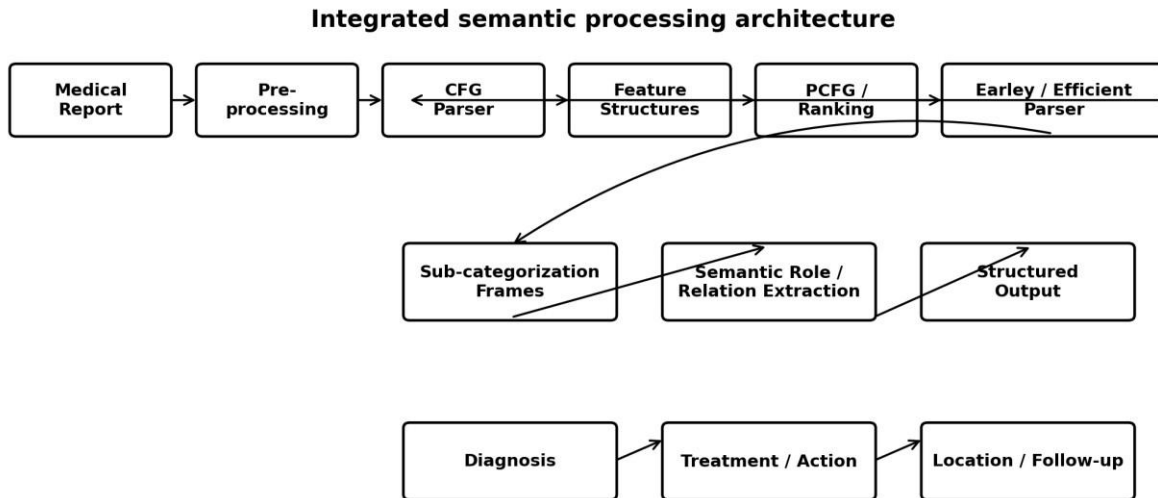
Conclusion

For this industry use case, Earley parsing is preferable when ambiguity, partial input and complex grammatical structures are important. Top-down parsing can remain useful for simpler, tightly controlled command grammars.

Q3. Healthcare NLP – Integrated Semantic Processing Architecture

The healthcare scenario requires extraction of diagnosis, prescribed treatments and follow-up actions from complex medical reports. The supplied example is: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.” The current basic CFG system fails to correctly interpret relationships and produces inconsistent outputs.

(a) Complete NLP architecture



The proposed architecture integrates the exact components requested in the assessment: CFG, PCFG, feature structures, efficient parsing, sub-categorization frames and semantic extraction.

Layer	Technique	Role in the system
Input & preprocessing	Tokenization, normalization, sentence handling	Prepare medical report text
Syntactic structure	CFG	Generate grammatical structures
Agreement handling	Feature Structures	Track person/number and grammatical attributes
Ambiguity resolution	PCFG	Rank competing parse structures
Efficient parsing	Earley / chart parsing	Handle lengthy and complex sentences efficiently
Verb semantics	Sub-categorization frames	Represent which entities/arguments medical verbs expect
Semantic extraction	Entity + relation/role extraction	Connect doctors, patients, actions, treatments and locations
Structured output	Clinical information schema	Return diagnosis,

		treatment/action, location and follow-up information
--	--	--

(b) Step-by-step workflow for the given medical sentence

4. Input: Receive the medical sentence from the hospital information system.
5. Pre-processing: tokenize the sentence and normalize medical text while preserving clinically meaningful terms.
6. CFG parsing: construct candidate syntactic structures for the sentence.
7. Feature checking: use feature structures to verify grammatical agreement and maintain relevant attributes.
8. PCFG ranking: assign probabilities to competing parses and select the most plausible structure.
9. Efficient parsing: use chart/Earley-style processing to reuse partial structures and avoid unnecessary recomputation.
10. Sub-categorization: apply the argument patterns of verbs such as “reviewed” and “recommends” to determine expected entities and complements.
11. Semantic extraction: identify the doctor, patient, medication action, follow-up action and Chennai as the location.
12. Structured output: convert the extracted information into a machine-readable representation.

Structured interpretation of the supplied example

Doctor/Agent: the doctor. Patient/Entity: the patient. Medical action: starting medication. Follow-up action: scheduling a follow-up visit. Location: Chennai. The sentence itself does not explicitly provide a diagnosis, so the output should not invent one.

Suggested structured output

Field	Extracted information
Agent	Doctor
Patient/Entity	Patient
Diagnosis	Not explicitly stated in the supplied sentence
Treatment action	Start medication
Follow-up action	Schedule a follow-up visit
Location	Chennai

(c) Method for sub-categorization, real-time processing, accuracy and scalability

1. Sub-categorization frames for medical verbs

Medical verbs can be represented with frames describing the arguments they typically take. For example, “recommend” can be represented as an action involving a medical professional as the Agent and a treatment or follow-up action as the recommended complement.

Conceptual frame:

RECOMMEND frame

RECOMMEND(Agent, Recommended_Action) → recommend(doctor, start medication / schedule follow-up).

Similarly, “review” can be represented as an action involving a reviewer and a patient/report entity. These frames help the system preserve relationships that a basic CFG alone does not encode.

2. Real-time data processing

- Use streaming or event-driven ingestion for incoming hospital reports.
- Process sentences incrementally so completed structures do not wait for the entire document.
- Use efficient chart parsing to reuse intermediate states.
- Keep the semantic extraction pipeline modular so high-priority clinical information can be produced quickly.
- Use confidence scores and validation rules before information is committed to clinical systems.

3. Accuracy improvement

- Combine syntactic parsing with probabilistic ranking rather than relying only on CFG.
- Use feature structures for agreement and grammatical constraints.
- Use medical-domain sub-categorization frames.
- Use annotated clinical datasets for training and evaluation.
- Validate extracted relationships against domain rules.
- Preserve uncertainty when the sentence does not explicitly state a fact.

4. Scalability in a hospital environment

- Use modular services so preprocessing, parsing and semantic extraction can scale independently.
- Cache reusable grammar and intermediate resources.
- Use efficient chart parsing for long sentences.
- Batch non-urgent historical reports while prioritizing real-time clinical input.
- Monitor latency, parsing accuracy and extraction consistency as operational metrics.

Final architecture recommendation

CFG provides the syntactic foundation; feature structures enforce agreement; PCFG resolves competing interpretations; Earley/chart parsing improves efficient processing; sub-categorization frames preserve medical verb–entity relationships; semantic extraction produces structured clinical information.

OVERALL CONCLUSION

The three industry scenarios show that a basic CFG is valuable as a syntactic foundation but is insufficient by itself for realistic conversational, voice and healthcare NLP systems.

Question	Core Problem	Recommended Solution
Q1 – Banking	Ambiguity, agreement and longquery parsing	CFG + Feature Structures + PCFG + Earley parsing

